Predicting CRM Sales Opportunities Using Machine Learning

CS 582 Machine Learning — Group 2

Hong Thai Phan - Nguyen Khanh An Tran - Hoang Thien Bao Bui

Date: October 2026

Abstract

This project predicts whether a Customer Relationship Management (CRM) sales opportunity will be Won or Lost. We joined four tables from the Maven Analytics CRM dataset and compared a Dummy control, Logistic Regression, Random Forest, a Multi-Layer Perceptron (MLP), and TabNet. To avoid learning from future information, we removed outcome fields and split data by engagement date, with a check that labels were available before training. On the original CRM data, Logistic Regression was selected with a test ROC-AUC of 0.524; the other models were also close to random ranking. We then tested the same pipeline on 100,000 simulated deals with a known win formula. TabNet reached 0.867 ROC-AUC, near the oracle score of 0.878. The system also reports predicted win probabilities and score explanations, while making clear that these results do not prove better real-world sales decisions.

1. Introduction

1.1. Predicting CRM sales opportunities

A sales team may have hundreds of deals in progress. Managers would like to know which opportunities have a higher chance of being Won. Machine learning can use past deals to produce a win probability for each open opportunity. In our project, a Won deal has target 1 and a Lost deal has target 0. Open deals do not have a final outcome, so we do not use them as labeled training examples.

1.2. Machine learning approach

We compare five classifiers rather than trusting only one algorithm. We also check whether the predicted probabilities can be explained in a useful way. The main contribution is the combination of a time-aware training process, leakage checks, explanations, input warnings, and a known-truth simulation. The project does not claim to invent a new machine learning algorithm.

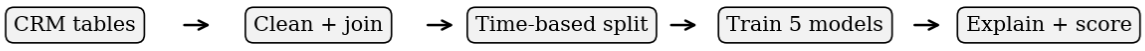


Fig. 1. Main workflow: preparation, temporal evaluation, model training, and scoring. Source: project code on main.

2. Dataset

2.1. Original CRM dataset

The dataset represents a fictitious business-to-business computer hardware company. It has four related CSV files: sales_pipeline (deals), accounts (customers), products (items for sale), and sales_teams (agents and managers). There are 8,800 opportunities, 85 accounts, 7 products, and 35 sales agents. The pipeline includes 4,238 Won, 2,473 Lost, 1,589 Engaging, and 500 Prospecting records.

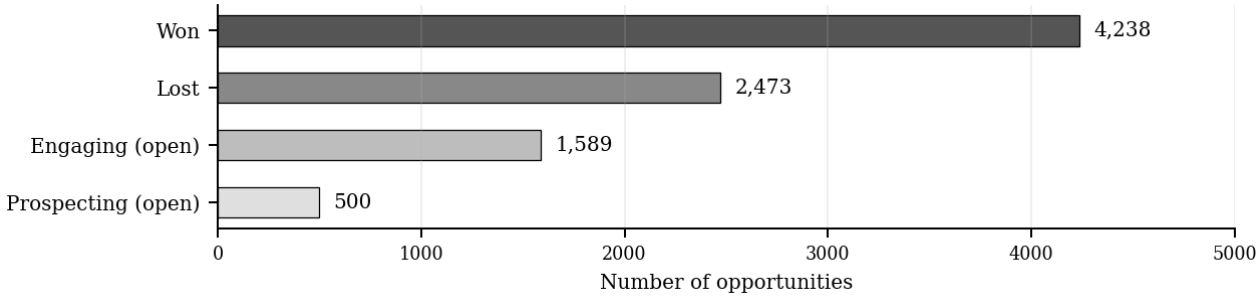


Fig. 2. Distribution of all 8,800 sales opportunities. Source: Group 2 presentation, slides 3–4, and main/data/crm/sales_pipeline.csv.

Only the 6,711 closed deals (Won or Lost) can be used as labeled examples. The 1,589 Engaging deals can be scored later. Prospecting deals have no engagement date and are not scored by the current pipeline.

2.2. Challenges with this dataset

The original data has limited information about customer communication and the progress of negotiations. Some open opportunities are missing an account. Also, a deal can close long after its engagement date, so its outcome may not have been known at an earlier training cutoff. Static account, product, and sales-team tables may not describe the exact values at that historical time. We handle some of these issues in data preparation, but not all can be solved with the available files.

2.3. Additional simulated dataset

To check whether our pipeline can learn meaningful patterns, we generated a separate dataset of 100,000 deals. It has a known formula for win probability, including product, company, industry, and competitor-related effects. We compare 18 standard CRM features with 29 features after adding 11 simulated inputs. These results are a controlled experiment, not results from real customers.

3. Data Preparation

3.1. Cleaning, joining, and creating features

We join the four CRM tables by product, account, and sales agent. The source code corrects spelling differences such as “GTXPro” to “GTX Pro” and “technolgy” to “technology.” It creates features such as the engagement month, quarter, day of week, and account age at engagement. Categorical values are one-hot encoded, missing values are imputed, and numeric values are scaled when needed. The preprocessing steps are fitted on training rows only.

3.2. Preventing data leakage

The features close_value, close_date, deal_stage, and opportunity_id are not used to make predictions. close_value is especially dangerous because Lost deals have a value of zero in the dataset. A one-rule decision tree using this column reaches ROC-AUC 1.000, but it has looked at the answer. We show this only as an invalid leakage example, not as a successful prediction model.

3.3. Splitting the data by time

We order closed opportunities by engagement date and keep all deals from the same date in one period. For training and validation, a deal is included only when its close date is before the next period starts. This removes 1,792 records with outcomes unavailable at the required cutoff. The split reported by the group is:

Period	Number of deals	Role
Train	2,975	Fit models and preprocessing
Validation	583	Choose model, threshold, and calibration
Test	1,361	Exploratory held-out evaluation
Purged	1,792	Outcome not available at cutoff

4. Training and Result

4.1. Training five machine learning models

The models are (1) Dummy Prior, which predicts the training win rate; (2) Logistic Regression, a linear baseline; (3) Random Forest with 300 trees and minimum leaf size 5; (4) MLP with two hidden layers of 64 and 32 neurons; and (5) TabNet for tabular data. We use a shared preprocessing pipeline and CPU execution with random seed 42. The best model is selected by validation ROC-AUC. A decision threshold is chosen using validation macro-F1, followed by sigmoid calibration on validation data.

4.2. Training results on the original CRM dataset

Logistic Regression had the best validation ROC-AUC of 0.563 and was selected before the test comparison. However, the selected model obtained only 0.524 on the test data. The results below are taken from the numbers the group reported in its final PowerPoint presentation.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Brier
Dummy prior	0.600	0.600	1.000	0.750	0.500	0.2421
Logistic Regression*	0.437	0.615	0.166	0.262	0.524	0.2794
Random Forest	0.590	0.598	0.966	0.739	0.519	0.2508
MLP	0.485	0.593	0.453	0.514	0.495	0.3314
TabNet	0.563	0.613	0.734	0.669	0.517	0.2468

* Model chosen on validation. These are the group-reported original (before-calibration) test metrics. The Dummy model gets 60% accuracy mainly because it always predicts Won, not because it ranks deals well.

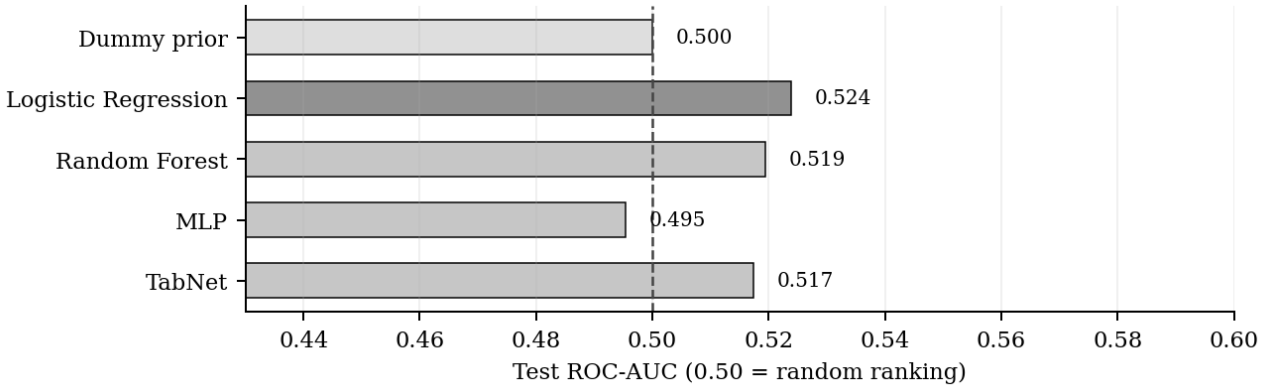


Fig. 3. Test ROC-AUC on original CRM data. Source: Group 2 final PowerPoint, slides 7–8.

All four learned models are around 0.50 to 0.52 ROC-AUC, so their ranking ability on this dataset is weak. Logistic Regression has low recall (0.166) at the threshold chosen on validation. Calibration improves its Brier score from 0.2794 to 0.2443, but the Dummy control is still slightly better at 0.2421. Calibration does not change ROC-AUC.

4.3. Model explanations and open-deal scoring

We compute permutation importance and local score explanations. The largest measured permutation ROC-AUC drop for the selected model is for sales_agent (0.0228), but the other drops are small. For a single deal, the program replaces one input with its training reference value (median or mode) and measures the change in P(Won). A separate Random Forest TreeSHAP check also verifies that explanation contributions add up to its prediction. These explanations describe the model score, not causes of a real sale.

The saved model scores 1,589 Engaging deals. It assigns each deal a win probability, a loss probability, a predicted class, a priority band, and positive and negative score factors. The report flags 1,088 deals with a missing account. The group presentation shows 400 High, 1,189 Medium, and 0 Low priority deals; these priority bands are unvalidated heuristics. An additional example estimates about $2.50 million for the open pipeline from probability times list price. This is illustrative, not a validated three-month revenue forecast.

4.4. Feature extraction and comparison with the original data

Feature extraction turns the raw CRM tables into one row of numeric and categorical inputs per deal. For both datasets we extract the same 18 standard features: the product, series, and sales price; the account sector, office location, revenue, employees, revenue per employee, year established, account age at engagement, and subsidiary flag; the sales agent, manager, and regional office; and the engagement year, month, quarter, and day of week.

For the simulated data we extract 11 further deal-level features from side tables that the original CRM does not contain. Industry fit and trend (4 features): a sector-match flag that is 1 when the product's target industry equals the account's sector, and the average growth, growth slope, and last-year growth of the account's industry over the five years before the engagement year. Product attributes (3 features): recycled-material percentage, expected longevity in years, and a made-in-USA flag. Competitor comparison (4 features): the price of the rival product offered against the deal, and the differences in recycled percentage, longevity, and made-in-USA between our product and the rival. These features are joined to each deal by opportunity ID, and together with the 18 standard features they form the 29 extended features.

To separate the effect of the data from the effect of the extra features, the table below uses the same 18 standard features, models, and pipeline on both datasets.

Model	Original CRM data (18 features)	Simulated data (18 features)
Dummy prior	0.500	0.500
Logistic Regression	0.524	0.784
Random Forest	0.518	0.840
MLP	0.495	0.838
TabNet	0.507	0.840
Known-formula oracle	—	0.878

Test ROC-AUC. Original CRM values are the group-reported test metrics from Section 4.2; simulated values come from main/data/crm_simulated/model_results.csv (one seed, interaction strength k = 3). The oracle uses the simulator's true win probability and is not available for the original data.

With identical features, every model is near chance (0.50) on the original data but reaches about 0.84 on the simulated data, within 0.04 of the oracle value 0.878. Because the features, models, and code are the same, the difference comes from the data: the original CRM fields carry little information about whether a deal is won, while the simulator deliberately links outcomes to product, sector, and sales agent. Adding the 11 extracted features raises simulated ROC-AUC by a further 0.024 to 0.029 (Section 4.5). This gain is partly built in, because the simulator uses the same variables to generate win probabilities. It shows that the models can use such information when a CRM records it, not that feature extraction would fix the original dataset, which does not have these fields.

4.5. Training results on simulated data

We also trained the models on simulated deals with known win probabilities. This helps us test whether the code can find a signal when one exists. Fig. 4 shows test ROC-AUC when models use the 18 standard features versus the 29 extended features. The oracle score, calculated using the true generating probabilities, is 0.878 and is never supplied to the models as an input.

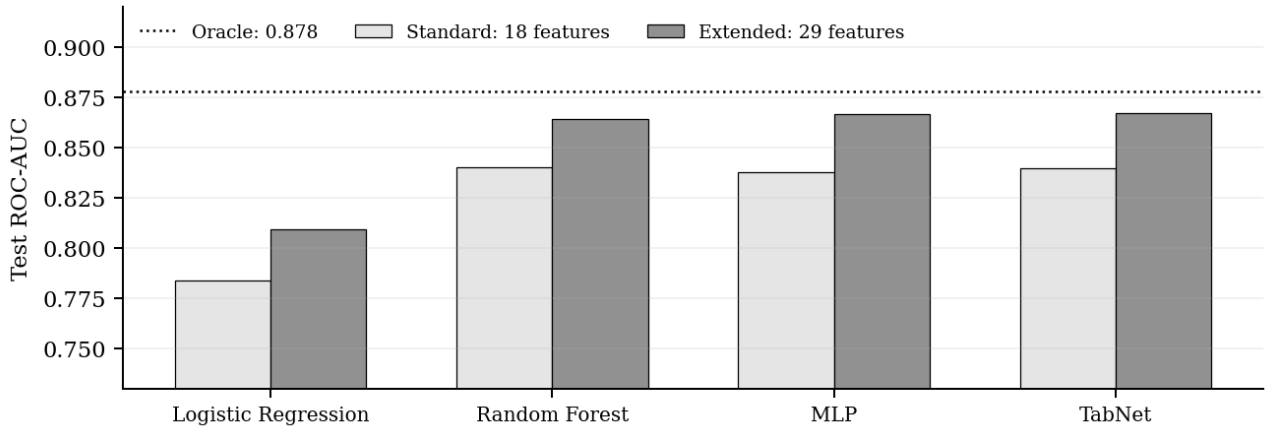


Fig. 4. Simulated test ROC-AUC, with known-truth oracle. Source: main/data/crm_simulated/model_results.csv and presentation slide 14.

For the extended simulated features, TabNet has a test ROC-AUC of 0.867, close to the oracle value 0.878. Five further generated datasets at interaction strength k = 3 reported mean ROC-AUC ± standard deviation of 0.875 ± 0.008 for TabNet, 0.873 ± 0.008 for MLP, 0.872 ± 0.007 for Random Forest, and 0.827 ± 0.008 for Logistic Regression. These are synthetic-data checks; they cannot be presented as accuracy on real deals.

4.6. Novelty, limitations, and lessons learned

Our novelty is in the experimental workflow, not in a new algorithm: removing outcome leakage; checking whether labels were available at each time cutoff; combining predictions with explanations and missing-input warnings; and using a known-truth simulator to diagnose the results. This is more useful than reporting a single model accuracy without checking how it was obtained.

The simulator separates a model problem from a data problem. The original CRM fields are not enough to predict deal outcomes: all models score near chance. With the same 18 features, models, and pipeline, the simulated data reaches about 0.84 ROC-AUC, within 0.04 of the oracle value 0.878, so the weak original results come from the data rather than from the models. Adding the 11 industry, product, and competitor features brings the best model to 0.867, within 0.011 of the oracle, which points to the information a real CRM should record. The simulated data is used only for this check; it is never used to train or improve the models applied to real deals.

Our original test is exploratory because earlier work inspected the dataset. Models are trained only on closed deals; accounts can repeat across periods; and validation is reused for several decisions. The CRM fields do not include calls, emails, budgets, or response times. Therefore the results do not show that the current system improves sales prioritization. Better future tests should use richer engagement data, genuine historical as-of scoring, unseen accounts, and externally validated priority thresholds.

5. Setup

5.1. Running the training and prediction application

The project runs with Python 3.12 and CPU-only dependencies. The README in the main branch provides these commands from the repository root:

Step	Command
1. Install	python3.12 scripts/tools/setup_cpu.py
2. Run full pipeline	.venv-crm/bin/python -m src.run_project
3. Train only	.venv-crm/bin/python -m src.train
4. Evaluate only	.venv-crm/bin/python -m src.evaluate
5. Score open deals	.venv-crm/bin/python -m src.predict

The full pipeline trains the models, evaluates them, and writes open-deal predictions to reports/crm/final/. The --quick option performs a smoke test and should not be used for reportable results. On Windows, use .venv-crm\Scripts\python.exe instead of .venv-crm/bin/python.

5.2. Reproducing the simulated experiment

For the stored simulated dataset, the README documents training with --data-dir data/crm_simulated and comparing models with scripts/simulation/run_simulated_models.py. Model comparison files and multi-seed summaries are stored under data/crm_simulated/. The original dataset metrics cited in this report are the numbers in the group final presentation, not an invented run log.

5.3. Using the prediction output

After training, the predict command writes open_deal_predictions.csv. A new input file may also be scored with .venv-crm/bin/python -m src.predict --input new_deals.csv --save new_deal_scores.csv. The input file needs the fields opportunity_id, sales_agent, product, account, and engage_date. The three original notebooks (1_train.ipynb, 2_evaluate.ipynb, and 3_predict.ipynb) provide another way to run the same stages.